

| MCU_FFT_speed_v5_followup_plan

| MCU 8 点 FFT speed_v5 后续规划

基线分支：`speed_v5_cnt160_95MHz`
对照数据：课程提供的“测试数据样例-2026”
规划目标：先兼容现场测试格式，再继续降低周期数或提高工作频率

| 1. 先回答最关键的问题

当前 `speed_v5_cnt160_95MHz` 不能直接使用课程测试样例得到正确结果。

不过，已经完成的两项速度优化是有价值的，可以继续保留：

- FFT 指令窗口已经由原 baseline 缩短到 `cnt_test = 160`。
- 时钟目标已经提高到 95 MHz。
- 第一阶段直接从 `test_ROM` 取数、最后阶段直接写 `verify_RAM` 的思路可以保留。

因此，下一步不是推倒重来，而是在 `speed_v5` 上建立一个“官方格式兼容版”，同时修正 FFT 蝶形流程。

| 2. 为什么当前结构不能直接运行官方样例

| 2.1 输入 ROM 容量和地址位宽不够

课程样例 `FFT_input.coe` 一共有 144 个 8 位数据：

地址	内容	定点格式
0~63	8×8 DFT 矩阵实部	Q7
64~127	8×8 DFT 矩阵虚部	Q7
128~135	8 个输入实部	Q5
136~143	8 个输入虚部	Q5

当前工程只配置了 16 深度的 `test_ROM`，并且有效地址只使用低 4 位，所以只能读取 0~15，无法访问 128~143 的 FFT 输入数据。

| 2.2 输入和输出的排列方式不同

当前程序使用交错布局：

实0，虚0，实1，虚1，...，实7，虚7

课程样例使用分区布局：

输入：
128~135 = 实部
136~143 = 虚部

输出：
0~7 = 实部
8~15 = 虚部

因此，即使 ROM 扩容，现有汇编仍会从错误地址读数，并把结果写到错误位置。

| 2.3 定点格式不同

当前程序主要按 Q15 运算，旋转因子使用 `23170 ≈ 0.7071 × 2^15`。

课程样例的矩阵旋转因子是 Q7：

$0.7071 \times 128 \approx 91$

样例输入是 Q5，输出是 Q12。为了得到与样例完全一致的整数结果，可以在读入时将 Q5 输入左移 7 位，使内部数据直接成为 Q12；乘以 Q7 旋转因子后再右移 7 位。

2.4 当前蝶形公式不符合官方自然序输入

当前程序的主要形式是：

```
t = b × W
y0 = a + t
y1 = a - t
```

但程序的分组顺序又是先做 (0,4)、(1,5) 等 DIF 分组。这种公式和分组的组合，与课程样例采用的自然序输入 DFT 不一致。

自然序输入的 8 点 radix-2 DIF 蝶形应为：

```
sum  = a + b
diff = a - b
y0   = sum
y1   = diff × W
```

按这个流程计算，并在最后进行位反转顺序整理，能够与课程样例的 16 个输出逐项完全一致。

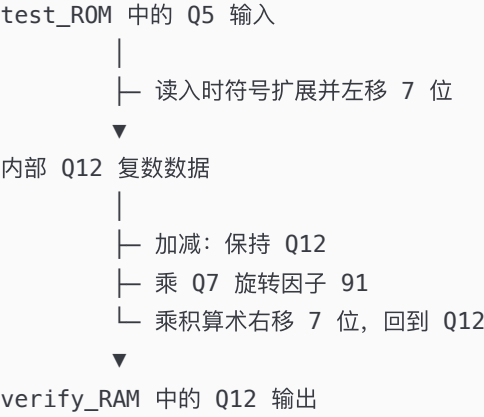
2.5 旧仿真通过不等于已经通过官方样例

旧检查脚本采用了与当前汇编相同的运算模型，所以能够证明 RTL 与原程序彼此一致，但不能证明它们与课程提供的 MATLAB/DFT 参考结果一致。

后续必须把“官方 .coe 的期望结果”作为最终判定标准。

3. 官方样例的正确数据通路

建议内部统一采用 Q12 数据：



这样既符合课程样例的定点定义，也不需要指令程序中增加 7 次移位操作。

4. 第一阶段：建立官方样例兼容版

建议新建分支：

```
speed_v6_official_sample
```

这一阶段只追求四件事：

- 1. 官方样例逐项完全正确。
- 2. cnt_test 能停止。

3. 尽量保持 `cnt_test = 160`。
4. 95 MHz 实现后 WNS 为正。

4.1 扩展 test_ROM

建议将 ROM 配置为：

```
数据宽度：8 bit
深度：256
地址宽度：8 bit
初始化文件：官方 FFT_input.coe
```

RTL 端的地址链路也统一扩展为 8 位：

- board_top
- mcu_top
- mcu_core
- ext_test_rom_if
- test_ROM IP wrapper

虽然样例只有 144 个数据，但使用 256 深度更适合 8 位地址，也方便后续现场替换 `.coe`。

4.2 按官方地址读取 8 个复数输入

汇编第一阶段应读取：

点	实部地址	虚部地址
x0	128	136
x1	129	137
x2	130	138
x3	131	139
x4	132	140
x5	133	141
x6	134	142
x7	135	143

按照第一层蝶形配对，推荐实际加载顺序：

```
(x0, x4), (x1, x5), (x2, x6), (x3, x7)
```

4.3 在存储器接口完成 Q5 到 Q12 转换

为了不增加 `cnt_test`，建议只对 128~143 地址的输入执行：

```
scaled_data = $signed(test_vector_in) <<< 7;
```

这属于外部测试数据格式适配，FFT 的加、减、乘和蝶形仍然全部由 MCU 指令完成。

需要在答辩前确认课程是否允许接口完成定点格式转换。如果要求所有移位也必须由 MCU 指令执行，则增加一条通用移位指令 `SHL`，但周期数会相应增加。

4.4 将乘法改成 Q7

旋转因子改为：

```
C707 = 91
```

乘法结果采用：

```
mul_q7 = ($signed(a) * $signed(b)) >>> 7;
```

第一版可先保留通用乘法器，确保结果正确；后面再优化乘法器宽度和 DSP 使用。

4.5 重写为正确的 DIF 蝶形

三级配对保持：

```
第 1 级: (0,4) (1,5) (2,6) (3,7)
第 2 级: (0,2) (1,3) (4,6) (5,7)
第 3 级: (0,1) (2,3) (4,5) (6,7)
```

每个蝶形统一改为：

```
sum  = a + b
diff = a - b
upper = sum
lower = diff × W
```

因为加减、装载、存储和乘法数量与现有程序接近，重新排布寄存器后，仍有较大机会保持 165 条左右的程序长度，即 `cnt_test ≈ 160`。

4.6 按官方格式写入 `verify_RAM`

DIF 计算结果是位反转顺序，最终频点映射为：

```
内部顺序: 0, 4, 2, 6, 1, 5, 3, 7
目标顺序: 0, 1, 2, 3, 4, 5, 6, 7
```

写入地址必须是：

```
verify_RAM[0~7] = X0~X7 实部
verify_RAM[8~15] = X0~X7 虚部
```

最后一次写操作应落在地址 15，使 `cnt_test` 在完整输出后停止。

5. 第一阶段的验收顺序

不要一开始就直接跑实现。建议按下面顺序逐层排错：

A. 软件参考模型

- 读取官方 144 字 `.coe`。
- 采用正确 DIF 蝶形和 Q5/Q7/Q12 定点规则。
- 确认 16 个输出与 `FFT_output.coe` 完全一致。

B. RTL 仿真

- 检查实际读取地址是否为 128~143。
- 检查读入 MCU 后的数据是否已经变成 Q12。
- 检查每一级蝶形的中间值。
- 检查 `verify_RAM[0:15]`。
- 检查 `cnt_test` 最终是否为 160，并保持停止。

C. Vivado 综合与实现

- 使用 95 MHz 时钟约束。
- Timing Summary 中 `WNS > 0`。
- 保存 LUT、FF、DSP、BRAM 和 WNS 数据。
- 不将 ILA 资源计入最终比赛版本。

D. 上板

- 用官方样例 `.coe` 生成 bitstream。
- 用 ILA 观察输入地址、输出写使能、输出地址、输出数据和 `cnt_test`。
- 验证通过后建立无 ILA 的提交版本。

6. 推荐测试集合

仅通过一个官方样例还不够。建议准备以下测试：

测试	目的
官方 2026 样例	验证文件格式和最终整数结果
全零输入	检查初始化和残留状态
单位脉冲	所有频点应有相同输出
单频复指数	检查频点顺序和实虚部符号
最大正负值	检查溢出、符号扩展和乘法位宽
至少 100 组随机输入	与软件定点模型逐项比较

每次更改汇编、ALU 或存储接口后，都要自动运行这些测试。

7. 通过样例后的速度优化路线

95 MHz、`cnt_test=160` 时，理论计算时间约为：

160 / 95 MHz = 1.684 μs

速度优化应分成两条独立路线，避免同时修改过多模块后难以定位错误。

路线 A：保持 160 周期，提高频率

A1. 缩窄乘法器

当前 32×32 Q15 乘法远大于实际需要。官方格式下，一个操作数主要是 Q12 数据，另一个旋转因子只有 8 位。

可改为：

数据有效宽度 × 8 位常数

收益：

- 缩短乘法组合路径。
- 减少 DSP 或 LUT。
- 为提高时钟频率留出余量。

A2. 常数乘 91

旋转因子只有 `±1`、`±j` 和 `±(91/128 ± j91/128)`，不需要完整通用乘法器。

91 可以分解为：

91 = 64 + 16 + 8 + 2 + 1

或：

91 = 128 - 32 - 4 - 1

可用移位和加减实现固定常数乘法，并比较以下两个版本：

- `use_dsp = yes`：优先追求频率。
- `use_dsp = no`：优先消除 DSP，观察 LUT 和 WNS。

不要只凭“用了几个 DSP”判断优劣，要同时记录 WNS、LUT、FF 和最终速度。

A3. Vivado 实现策略

建议对同一 RTL 批量比较：

- `Performance_Explore`
- `Performance_ExplorePostRoutePhysOpt`
- `Congestion_SpreadLogic_high`
- `phys_opt_design` 开启
- retiming 开启与关闭两组

这些设置可能改善布局布线，但不能替代 RTL 关键路径优化。每组设置都要保存独立结果，避免只看一次运行。

A4. 逐级提高频率

推荐测试顺序：

100 MHz → 110 MHz → 120 MHz → 130 MHz

每一级必须满足：

- WNS 为正。
- 官方样例和随机仿真仍通过。
- 上板运行稳定。

理论时间参考：

频率	160 周期计算时间
95 MHz	1.684 μs
110 MHz	1.455 μs
120 MHz	1.333 μs
130 MHz	1.231 μs

路线 B：保持可实现频率，减少周期数

B1. 复数装载和存储指令

当前一个复数需要两次读取或写入。可增加：

LDC：一次加载实部和虚部
STC：一次写回实部和虚部

这要求 MCU 内部提供成对寄存器或双端口存储访问。先评估新增控制逻辑是否会降低频率。

B2. 复数加减指令

增加面向两个寄存器对的：

CADD

一条指令同时完成实部和虚部运算，可减少大量标量 ADD/SUB 指令。

B3. DIF 蝶形指令

在 MCU ISA 中增加通用蝶形指令：

```
BFY rd_a, rd_b, twiddle
```

该指令完成一个 radix-2 蝶形，但仍由 MCU 取指、译码和执行，FFT 仍然是指令程序，而不是绕开 MCU 的独立 FFT 核。

这条路线需要提前向教师确认规则边界。建议只做“通用复数蝶形指令”，不要设计“一条指令完成整个 8 点 FFT”。

B4. 简单流水线

只有在明确找到关键路径后再加入流水线，例如：

- 取指与执行分离。
- 乘法分两级。
- 写回与下一条取指重叠。

流水线会引入冒险、停顿和旁路逻辑。对于只有约 160 周期的小程序，必须用实际结果证明新增 FF 没有抵消速度收益。

B5. 多核放到最后

8 点 FFT 规模很小，多核会增加：

- 指令存储器和寄存器堆。
- 输入分配与结果合并逻辑。
- 同步等待周期。
- LUT 和 FF。

在单个 FFT 任务中，多核通常不如专门优化 MCU 指令和数据通路划算。只有赛题允许并行处理多个独立 FFT 数据块时，多核才值得优先考虑。

8. 推荐分支和里程碑

分支	目标	进入下一阶段的条件
speed_v5_cnt160_95MHz	保留当前成果	冻结，不再直接改
speed_v6_official_sample	官方格式兼容、正确 DIF	官方样例精确通过，cnt_test≈160
speed_v7_q7_narrow_mul	缩窄乘法器、优化 WNS	结果不变，WNS 或资源明显改善
speed_v8_high_freq	提高到 110~130 MHz	WNS 为正，上板稳定
speed_v9_cycle_reduce	增加复数/蝶形指令	周期显著下降且综合收益为正

每个分支都应提交：

- RTL 和汇编源码。
- 生成后的指令 ROM。
- 官方样例仿真结果。
- 随机回归结果。
- cnt_test。
- 频率和 WNS。
- LUT、FF、DSP、BRAM。

9. 建议优先级

接下来最合理的顺序是：

- 冻结 speed_v5，不要在唯一可用版本上继续叠加修改。

2. 扩展 ROM 和地址位宽，接入官方 144 字格式。
3. 修正 DIF 蝶形和 Q 格式，先让官方样例精确通过。
4. 重新汇编并确认能否保持 `cnt_test=160`。
5. 在 95 MHz 下完成仿真、实现和上板验证。
6. 缩窄乘法器并尝试 110~130 MHz。
7. 再考虑复数指令、蝶形指令和流水线来减少周期。
8. 多核仅作为处理多数据块时的后备方案。

10. 阶段目标表

阶段	正确性	周期目标	频率目标	重点
当前 <code>speed_v5</code>	旧格式通过	160	95 MHz	已完成的速度基线
官方兼容版	官方样例精确通过	160 左右	95 MHz	ROM、Q 格式、DIF、输出布局
高频版	官方及随机测试通过	160	110~130 MHz	缩窄/常数乘法、关键路径
低周期版	官方及随机测试通过	80~120	视实现结果	复数指令、蝶形指令、流水线

最终成绩必须按真实实现结果计算，不能只比较时钟频率或指令条数。每次优化都应同时记录：

- `总时间 = cnt_test / 实际工作频率`
- `硬件资源成本 = 6 × LUT + 10 × FF`

在官方样例正确、WNS 为正的前提下，再比较哪个版本更适合速度榜。